

CS454PC - NODE JS/ REACT JS/ DJANGO LABORATORY MANUAL

**B.TECH. (R22)
(II YEAR – II SEM)**

**Prepared By
Ramakrishna Narla
(Asst. Prof.)
CSE Dept.**



**DEPARTMENT OF
COMPUTER SCIENCE AND ENGINEERING
MAHATMA GANDHI INSTITUTE OF TECHNOLOGY
GANDIPET, HYDERABAD, TS - 500075.**

L	T	P	C
0	0	2	1

IV Semester Syllabus
CS454PC: NODE JS/ REACT JS/ DJANGO
[Common to CSE, IT]

Prerequisites:

- 1.Object Oriented Programming through Java, HTML Basics.

Course Objectives

To Learn

- To implement the static web pages using HTML and do client side validation using JavaScript.
- To design and work with databases using Java
- To develop an end to end application using java full stack.
- To introduce Node JS implementation for server side programming.
- To experiment with single page application development using React.

Course Outcomes

Student will be able to:

- Build a custom website with HTML, CSS, and Bootstrap and little JavaScript.
- Demonstrate Advanced features of JavaScript and learn about JDBC
- Develop Server – side implementation using Java technologies like
- Develop the server – side implementation using Node JS.
- Design a Single Page Application using React..

Exercises:

1. Build a responsive web application for shopping cart with registration, login, catalog and cart pages using CSS3 features, flex and grid.
2. Make the above web application responsive web application using Bootstrap framework.
3. Use JavaScript for doing client – side validation of the pages implemented in experiment 1 and experiment 2.
4. Explore the features of ES6 like arrow functions, callbacks, promises, async/await. Implement an application for reading the weather information from openweathermap.org and display the information in the form of a graph on the web page.
5. Develop a java standalone application that connects with the database (Oracle / mySql) and perform the CRUD operation on the database tables.
6. Create an xml for the bookstore. Validate the same using both DTD and XSD.
7. Design a controller with servlet that provides the interaction with application developed in experiment 1 and the database created in experiment 5.
8. Maintaining the transactional history of any user is very important. Explore the various session tracking mechanism (Cookies, HTTP Session)
9. Create a custom server using http module and explore the other modules of Node JS like OS, path, event.
10. Develop an express web application that can interact with REST API to perform CRUD operations on student data. (Use Postman)
11. For the above application create authorized end points using JWT (JSON Web Token).
12. Create a react application for the student management system having registration, login, contact, about pages and implement routing to navigate through these pages.
13. Create a service in react that fetches the weather information from openweathermap.org and the display the current and historical weather information using graphical representation using chart.js
14. Create a TODO application in react with necessary components and deploy it into github.

Reference Books:

1. Jon Duckett, Beginning HTML, XHTML, CSS, and JavaScript, Wrox Publications, 2010
2. Bryan Basham, Kathy Sierra and Bert Bates, Head First Servlets and JSP, O'Reilly Media, 2nd Edition, 2008.
3. Vasan Subramanian, Pro MERN Stack, Full Stack Web App Development with Mongo, Express, React, and Node, 2 nd Edition, A Press.

Experiment # 1

Basic HTML with non-interactive elements like headings, paragraph, table, images, lists

Aim –

Get familiarized with basic static web pages

Sample code –

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Basic HTML Page</title>
</head>
<body>

  <!-- Headings -->
  <h1>Main Heading</h1>
  <h2>Subheading</h2>
  <h3>Section Heading</h3>

  <!-- Paragraph -->
  <p>This is a simple paragraph describing the content of this page. HTML is great for structuring web content.</p>

  <!-- Image -->
  

  <!-- Unordered List -->
  <h3>Unordered List of Fruits</h3>
  <ul>
    <li>Apple</li>
    <li>Banana</li>
    <li>Cherry</li>
  </ul>

  <!-- Ordered List -->
  <h3>Ordered List of Steps</h3>
  <ol>
    <li>Open the browser</li>
    <li>Type the URL</li>
    <li>Press Enter</li>
  </ol>

  <!-- Table -->
  <h3>Sample Table</h3>
  <table border="1">
    <tr>
      <th>Name</th>
      <th>Age</th>
      <th>Country</th>
    </tr>
    <tr>
      <td>Alice</td>
      <td>25</td>
      <td>USA</td>
    </tr>
    <tr>
      <td>Bob</td>
      <td>30</td>
      <td>UK</td>
    </tr>
  </table>

</body>
</html>
```

Main Heading

Subheading

Section Heading

This is a simple paragraph describing the content of this page. HTML is great for structuring web content.

 Placeholder Image

Unordered List of Fruits

- Apple
- Banana
- Cherry

Ordered List of Steps

1. Open the browser
2. Type the URL
3. Press Enter

Sample Table

Name	Age	Country
Alice	25	USA
Bob	30	UK

Experiment # 2

Basic HTML with interactive elements like input elements, buttons, drop down lists, anchor elements

Aim –

Get familiarized with basic web pages involving interactive UI elements

Sample code –

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Interactive HTML Page</title>
</head>
<body>

  <h1>Interactive Elements</h1>

  <!-- Anchor Link -->
  <p>Visit <a href=".\\page1.html" target="_blank">Another WebPage</a></p>

  <!-- Form with Input Elements -->
  <h2>Contact Form</h2>
  <form>
    <!-- Text Input -->
    <label for="name">Name:</label>
    <input type="text" id="name" name="name"><br><br>

    <!-- Email Input -->
    <label for="email">Email:</label>
    <input type="email" id="email" name="email"><br><br>

    <!-- Password Input -->
    <label for="password">Password:</label>
    <input type="password" id="password" name="password"><br><br>

    <!-- Radio Buttons -->
    <p>Choose your gender:</p>
    <input type="radio" id="male" name="gender" value="male">
    <label for="male">Male</label><br>

    <input type="radio" id="female" name="gender" value="female">
    <label for="female">Female</label><br><br>

    <!-- Checkbox -->
    <input type="checkbox" id="subscribe" name="subscribe">
    <label for="subscribe">Subscribe to newsletter</label><br><br>

    <!-- Dropdown -->
    <label for="country">Select your country:</label>
    <select id="country" name="country">
      <option value="usa">USA</option>
      <option value="uk">UK</option>
      <option value="canada">Canada</option>
    </select><br><br>

    <!-- Submit Button -->
    <button type="submit">Submit</button>
  </form>

</body>
</html>
```

Interactive Elements

Visit [Another WebPage](#)

Contact Form

Name:

Email:

Password:

Choose your gender:

- ☐ Male
☐ Female

☐ Subscribe to newsletter

Select your country: ▼

Experiment # 3

HTML with Inline CSS

Aim –

Get familiarized with style aspects of web pages, with inline CSS

Sample code –

```
<!DOCTYPE html>
<html>
<head>
  <title>Inline CSS Example</title>
</head>
<body>

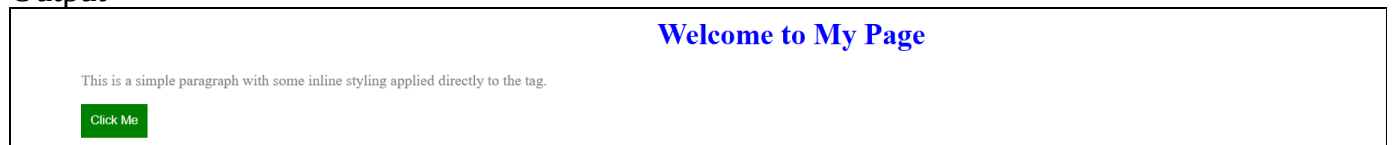
  <h1 style="color: blue; text-align: center;">Welcome to My Page</h1>

  <p style="font-size: 16px; color: gray;">
    This is a simple paragraph with some inline styling applied directly to the tag.
  </p>

  <button style="background-color: green; color: white; padding: 10px; border: none;">
    Click Me
  </button>

</body>
</html>
```

Output –



Experiment # 4

HTML with Internal CSS, CSS Selectors, Box model

Aim –

Get familiarized with Internal CSS styling, involving various CSS selectors.

Padding and Margins.

Sample code –

```
<!DOCTYPE html>
<html>
<head>
  <title>Internal CSS with Selectors</title>
  <style>
    /* Element selector */
    h1 {
      color: blue;
      text-align: center;
    }

    /* Class selector */
    .highlight {
      color: gray;
      font-size: 16px;
    }

    /* ID selector */
    #main-btn {
      background-color: green;
      color: white;
      padding: 10px;
      border: none;
    }

    /* Group selector */
    h1, p {
      font-family: Arial, sans-serif;
    }

    /* Descendant selector */
    .container p {
      margin-left: 20px;
    }
  </style>
</head>
<body>

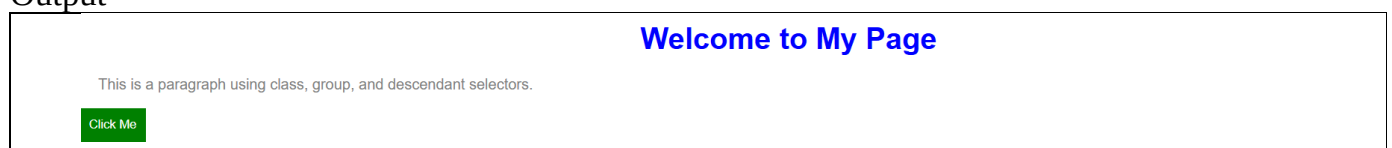
  <h1>Welcome to My Page</h1>

  <div class="container">
    <p class="highlight">This is a paragraph using class, group, and descendant selectors.</p>
  </div>

  <button id="main-btn">Click Me</button>

</body>
</html>
```

Output –



Experiment # 5

HTML with External CSS, CSS Selectors

Aim –

Get familiarized with External CSS styling, involving various CSS selectors.

Sample code –

CSS file:

```
/* Element selector */
h1 {
  color: blue;
  text-align: center;
}

/* Class selector */
.highlight {
  color: gray;
  font-size: 16px;
}

/* ID selector */
#main-btn {
  background-color: green;
  color: white;
  padding: 10px;
  border: none;
}

/* Group selector */
h1, p {
  font-family: Arial, sans-serif;
}

/* Descendant selector */
.container p {
  margin-left: 20px;
}
```

HTML file:

```
<!DOCTYPE html>
<html>
<head>
  <title>Internal CSS with Selectors</title>
  <link rel="stylesheet" href="styles.css"></head>
<body>

  <h1>Welcome to My Page</h1>

  <div class="container">
    <p class="highlight">This is a paragraph using class, group, and descendant selectors.</p>
  </div>

  <button id="main-btn">Click Me</button>

</body>
</html>
```

Output –

Welcome to My Page

This is a paragraph using class, group, and descendant selectors.

Click Me

Experiment # 6

Online shopping cart web application with catalog and cart pages, with CSS grid and flex layouts

Aim –

1. Practical usage of html.
2. CSS layouts – grid and flex.

Sample code –

```
ShoppingPage1.html:

<!DOCTYPE html>
<html lang="en">
<head>
  <title>OnlineStore</title>
  <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0">

  <style>
    .OverallContainer
    {
      display:grid;
      grid-template-rows: 60px auto 15px;
      height: 95vh;
      gap: 10px;
    }

    .HeaderSection
    {
      grid-row:1;

      display: grid;
      grid-template-columns: auto 25%;
      width: 100%;
      gap: 10px;
    }

    .Heading
    {
      grid-column:1;

      align-items: center;
      color:red;
    }

    .UserInfo
    {
      grid-column:2;

      display: flex;
      flex-direction: row;
      justify-content: flex-end;
      align-items: center;
      gap: 30px;
      padding-right: 30px;
    }

    .MainContent
    {
      grid-row:2;

      font-size:small;
      color:orange;
      background-color:lightyellow;
      padding:15px;
```

```

    }

    table {
        border-spacing: 10px; /* Set the gap between columns */
    }
    th, td {
        padding-left: 50px;
        <!-- border: 1px solid black; /* Optional: Adds borders to the cells */ -->
    }

    button
    {
        background-color:yellow;
        color:black;
        font-size:medium;
    }

    .FooterSection
    {
        grid-row:3;

        display:flex;
        flex-direction: row;
        align-items: center;
    }
</style>

<script>
    function addMobile()
    {

    }

    function addLaptop()
    {

    }

    function addTV()
    {
        if (document.getElementById("TVList1").value == "")
        {
            alert('Select an Item first');
        }
        else
        {
            localStorage.setItem("TV", document.getElementById("TVList1").value);
        }
    }
</script>
</head>
<body>
    <div class='OverallContainer'>

        <div class='HeaderSection'>
            <div class='Heading'>
                <h1>Online Store</h1>
            </div>
            <div class='UserInfo'>
                <A href='ShoppingPageCart.html'>Cart</A>
                <A href='.\MeProfile'>Me</A>
            </div>
        </div>

        <div class='MainContent'>
            <table class='MainTable'>

```

```

<tr>
  <td>
    
  </td>
  <td>
    <label>Mobiles</label><br>
    <select name='ItemList' id='MobileList1'>
      <option value=" selected">Select a Phone</option>
      <option value='SamsungS25'>Samsung S25</option>
      <option value='OnePlusNord'>OnePlus Nord</option>
      <option value='iPhone'>iPhone 15</option>
      <option value='GooglePixel'>Google Pixel 5</option>
    </select>
  </td>
  <td>
    <button class='btn btn-primary' onClick="addMobile()">Add to Cart</button>
  </td>
</tr>
<tr>
  <td>
    
  </td>
  <td>
    <label>Laptops</label><br>
    <select name='ItemList' id='LaptopList1'>
      <option value=" selected">Select a Laptop</option>
      <option value='HPblah'>HP Model blah</option>
      <option value='LenovoBlah'>Lenovo Model blah</option>
      <option value='AcerBlah'>Acer Model blah</option>
    </select>
  </td>
  <td>
    <button class='btn btn-primary' onClick="addLaptop()">Add to Cart</button>
  </td>
</tr>
<tr>
  <td>
    
  </td>
  <td>
    <label>TeleVision</label><br>
    <select name='ItemList' id='TVList1'>
      <option value=" selected">Select a TV</option>
      <option value='Samsung'>Samsung</option>
      <option value='LG'>LG</option>
      <option value='Sony'>Sony</option>
    </select>
  </td>
  <td>
    <button class='btn btn-primary' onClick="addTV()">Add to Cart</button>
  </td>
</tr>
</table>
</div>

<div class='FooterSection'>
(c) Online Store, Reg # 567890, Gandipet, Hyderabad, TG.
</div>

</div> <!-- OverallContainer -->
</body>
</html>

```

ShoppingPageCart.html

```

<!DOCTYPE html>
<html lang="en">

```

```

<head>
<title>OnlineStore</title>
<meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0">

<style>
.OverallContainer
{
    display:grid;
    grid-template-rows: 60px auto 15px;
    height: 95vh;
    gap: 10px;
}

.HeaderSection
{
    grid-row:1;

    display: grid;
    grid-template-columns: auto 25%;
    width: 100%;
    gap: 10px;
}

.Heading
{
    grid-column:1;

    align-items: center;
    color:red;
}

.UserInfo
{
    grid-column:2;

    display: flex;
    flex-direction: row;
    justify-content: flex-end;
    align-items: center;
    gap: 30px;
    padding-right: 30px;
}

.MainContent
{
    grid-row:2;

    font-size:small;
    color:orange;
    background-color:lightyellow;
    padding:15px;
}

button
{
    background-color:yellow;
    color:black;
    font-size:medium;
}

td
{
    padding: 20px;
}

.FooterSection
{

```

```

        grid-row:3;

        display:flex;
        flex-direction: row;
        align-items: center;
    }
</style>
</head>
<body>
    <div class='OverallContainer'>

        <div class='HeaderSection'>
            <div class='Heading'>
                <h1>Online Store</h1>
            </div>
            <div class='UserInfo'>
                <A href='.\Cart'>Cart</A>
                <A href='.\MeProfile'>Me</A>
            </div>
        </div>

        <div class='MainContent'>
            <table class='MainTable' >
                <tr style="display: none;">
                    <td>
                        
                    </td>
                    <td>
                        <button class='btn btn-primary' onClick="addMobile()">Remove from Cart</button>
                    </td>
                </tr>
                <tr style="display: none;">
                    <td>
                        
                    </td>
                    <td>
                        <button class='btn btn-primary' onClick="addLaptop()">Remove from Cart</button>
                    </td>
                </tr>
                <tr id="Row3" style="display: none;">
                    <td>
                        
                    </td>
                    <td>
                        <label id="SelectedTV1"></label>
                    </td>
                    <td>
                        <button class='btn btn-primary' onload="displayTV()">Remove from Cart</button>
                    </td>
                </tr>
            </table>

            <script>
                if (localStorage.getItem('TV') != "")
                {
                    document.getElementById('Row3').style.display = "";
                    document.getElementById('SelectedTV1').textContent = localStorage.getItem('TV');
                }
            </script>
        </div>
    </div>

```

```
<div class='FooterSection'>  
(c) Online Store, Reg # 567890, Gandipet, Hyderabad, TG.  
</div>
```

```
</div> <!-- OverallContainer -->  
</body>  
</html>
```

Output –

Online Store



Mobiles
Select a Phone ▾

Add to Cart



Laptops
Select a Laptop ▾

Add to Cart



TeleVision
Select a TV ▾

Add to Cart

(c) Online Store, Reg # 567890, Gandipet, Hyderabad, TG.

Online Store



LG

Remove from Cart

(c) Online Store, Reg # 567890, Gandipet, Hyderabad, TG.

Experiment # 7

Bootstrap - Online shopping cart web application with catalog and cart pages, with CSS grid and flex layouts

Aim –

1. Practical usage of html.
2. Familiarize with Bootstrap for more promising and responsive UI.

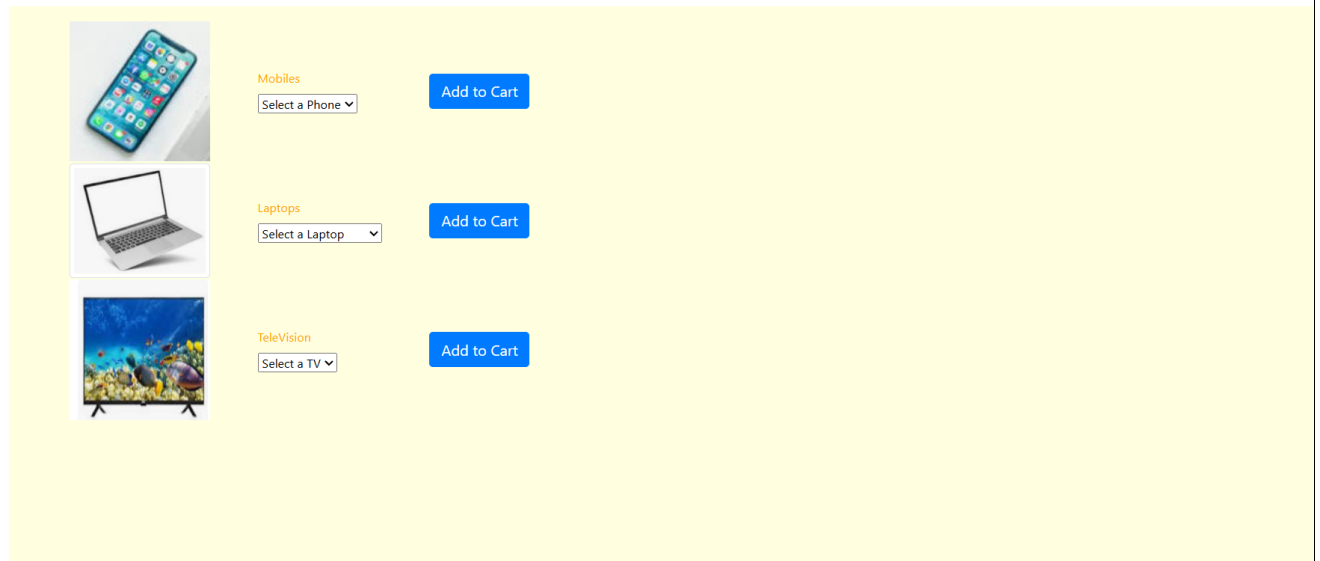
Sample code –

In the same above html page, include the following Bootstrap CSS link

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
```

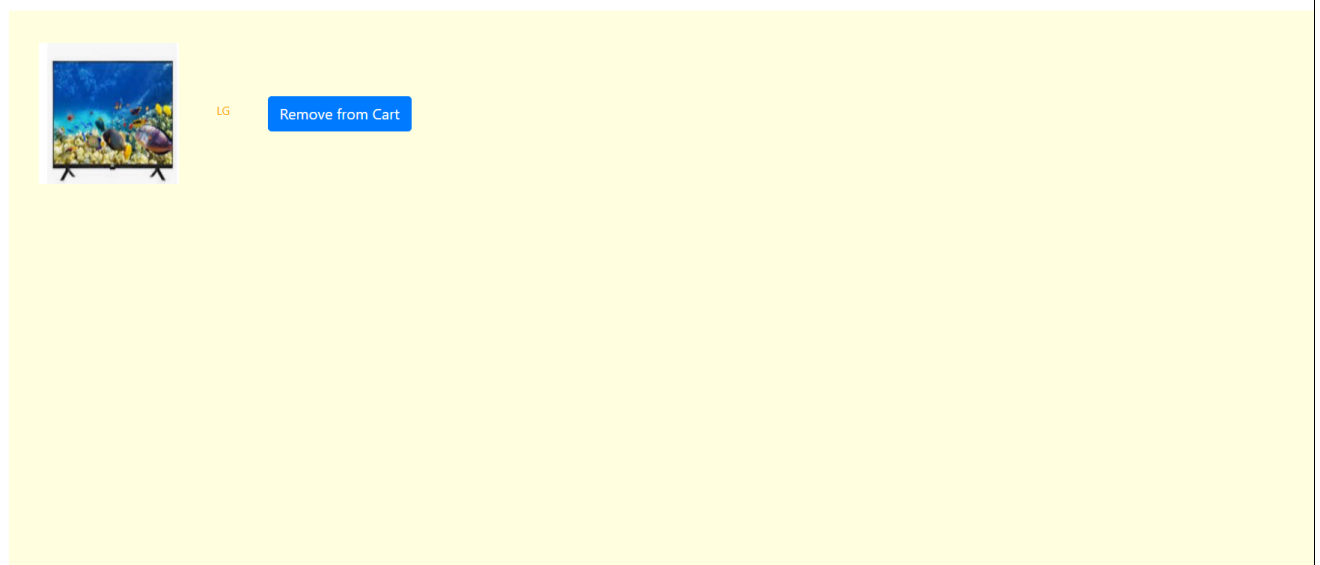
Output –

Online Store



(c) Online Store, Reg # 567890, Gandipet, Hyderabad, TG.

Online Store



(c) Online Store, Reg # 567890, Gandipet, Hyderabad, TG.

Observe more promising appearance of the buttons in UI.
Familiarize with Grid layout, using Bootstrap.

Some Viva questions on HTML, CSS and Bootstrap

HTML

1. Name any two interactive html elements.
2. What is the difference between head section and body section?
3. Which html element is used for logical grouping of html content?
4. What is the purpose of meta tag?
5. Which element is used for specifying styles? Internal CSS and External CSS.
6. Which element is used for having Javascript validations or functionalities? Internal JS and External JS. What is the purpose of having defer in script element?
7. How to have a table in a web page? Give html skeleton for it.
8. How to have a drop-down list in a web page? Give relevant html elements.

CSS

1. What is the purpose of CSS?
2. What are different ways to apply CSS to html?
3. What all CSS selectors are available? In what all ways, we can define or specify the styles to various html elements?
4. Explain CSS Box model of the elements. What is padding and what is margin? What all are configurable with them?
5. Name any two layouts supported in CSS. Brief on them. When to use which one?
6. How can we define a grid layout for any container? How to specify 3 rows and 2 columns.
7. What is flex-direction? How it helps in placing the elements?
8. When should we use grid layout and when flex layout?

Bootstrap

1. What is Bootstrap? What is its purpose?
2. How grid system is supported in Bootstrap (what is the class name to be used)? How many columns it assumes, by default. How to specify a row, in grid system (what is the class name to be used)?

Experiment # 8

Create User Account web page with Javascript validations embedded.

Aim –

Get familiarized with the core purpose of Javascript at client side.

Sample code –

```
<!DOCTYPE html>
<html>
  <head>
    <Title>Signin and Signup</Title>
    <script>
      function handleTheSubmit(event)
      {
        event.preventDefault();

        document.getElementById('UserNameId1Label').style.visibility = 'hidden';
        document.getElementById('PasswordId1Label').style.visibility = 'hidden';
        document.getElementById('dateId1Label').style.visibility = 'hidden';

        let regexForEmail = /^[a-zA-Z][a-zA-Z0-9]+@[a-zA-Z][a-zA-Z0-9]+\.[a-zA-Z]{2,}$/

        let userEmail = event.target.email1.value;
        if (!regexForEmail.test(userEmail))
        {
          document.getElementById('UserNameId1Label').style.visibility = 'visible';
        }

        let regexForUpperCase = /[A-Z]/
        let regexForLowerCase = /[a-z]/

        let passwd = event.target.password1.value;
        if (passwd.length < 8 || !regexForUpperCase.test(passwd) || !regexForLowerCase.test(passwd))
        {
          document.getElementById('PasswordId1Label').style.visibility = 'visible';
        }

        let dob = event.target.date1.value;
        if (new Date(dob) > new Date())
        {
          document.getElementById('dateId1Label').style.visibility = 'visible';
        }
      }
    </script>
  </head>
  <body>
    <form onsubmit="handleTheSubmit(event)">
      <label>UserName:</label>
      <input type="email" id="UserNameId1" name="email1"/>
      <br>
      <label id="UserNameId1Label" style="color:red; visibility:hidden;">*Invalid email format.</label>
      <br>
      <br>
      <label>Password:</label>
      <input type="password" id="PasswordId1" name="password1"/>
      <br>
      <label id="PasswordId1Label" style="color:red; visibility:hidden;">*High risk password. Please set
one including upper case, lower case and min 8 length.</label>
      <br>
      <br>
      <label>DoB:</label>
      <input type="date" id="dateId1" name="date1"/>
      <br>
      <label id="dateId1Label" style="color:red; visibility:hidden;">*Incorrect Date for DoB</label>
```

```
<br>
<br>
<input type="submit"/>
</form>
</body>
</html>
```

Output –

UserName:

Password:

***High risk password. Please set one including upper case, lower case and min 8 length.**

DoB: 

Integrate this with the above Online Shopping Cart web application, to give provision to the users to Sign up for their accounts.

Experiment # 9

User Login page with Javascript validations embedded.

Aim –

Get familiarized with the core purpose of Javascript at client side.

Sample code –

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Login Page</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f4;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
    }

    .login-box {
      background-color: white;
      padding: 30px;
      border-radius: 8px;
      box-shadow: 0 0 10px rgba(0,0,0,0.1);
      width: 300px;
    }

    .login-box h2 {
      text-align: center;
      margin-bottom: 20px;
    }

    .login-box input[type="text"],
    .login-box input[type="password"] {
      width: 100%;
      padding: 10px;
      margin: 10px 0;
      border: 1px solid #ccc;
      border-radius: 4px;
    }

    .login-box button {
      width: 100%;
      padding: 10px;
      background-color: #007BFF;
      color: white;
      border: none;
      border-radius: 4px;
      cursor: pointer;
    }

    .login-box a {
      display: block;
      text-align: center;
      margin-top: 15px;
      color: #007BFF;
      text-decoration: none;
    }
  </style>
</head>
<body>
```

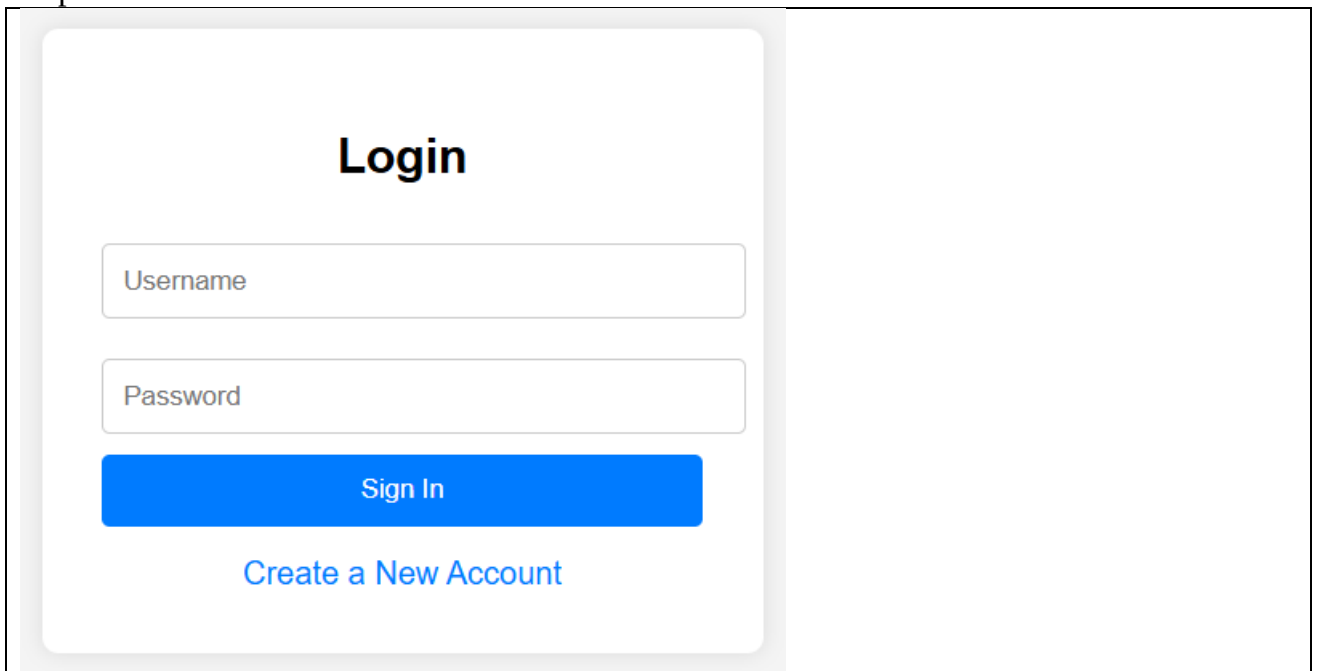
```
<div class="login-box">
  <h2>Login</h2>
  <input type="text" id="username" placeholder="Username">
  <input type="password" id="password" placeholder="Password">
  <button onclick="validateLogin()">Sign In</button>
  <a href="#">Create a New Account</a>
</div>

<script>
function validateLogin() {
  const username = document.getElementById('username').value.trim();
  const password = document.getElementById('password').value.trim();

  if (username === "" || password === "") {
    alert("Please enter both username and password.");
  }
}
</script>

</body>
</html>
```

Output –



Integrate this with the above Online Shopping Cart web application, to give provision to the users to Sign in to their accounts.

Experiment # 10

JavaScript Asynchronous Programming with Arrow Functions, Callbacks, Promises

Aim –

1. Understand the need for Asynchronous Programming.
2. Familiarize with Arrow Functions syntax and Callbacks.
3. How Promise with then, makes the code easy to follow, compared to callbacks?
4. How Promise with Async/Await makes the code even simpler!

Sample code –

Javascript code with Callback syntax -

```
<html lang="en">
<head>
  <script defer>

    // Simulate an asynchronous request to API 1
    function fetchDataFromAPI1(callback) {
      setTimeout(() => {
        console.log("Fetched data from API 1");
        callback(null, "Data from API 1");
      }, 2000);
    }

    // Simulate an asynchronous request to API 2
    function fetchDataFromAPI2(data1, callback) {
      setTimeout(() => {
        console.log("Fetched data from API 2, based on " + data1);
        callback(null, "Data from API 2, based on " + data1);
      }, 2000);
    }

    function fetchData()
    {
      // Fetch data sequentially using callbacks
      fetchDataFromAPI1((error1, data1) => {
        if (error1) {
          console.log("Error fetching data from API 1");
        } else {
          console.log(data1);
          fetchDataFromAPI2(data1, (error2, data2) => {
            if (error2) {
              console.log("Error fetching data from API 2");
            } else {
              console.log(data2);
              // Continue with other operations...
            }
          });
        }
      });
    }

  </script>
</head>
<body>
  <button onclick="fetchData()">ClickMe</button><br>
</body>
</html>
```

Javascript code with Promise, then & Async/Await -

```
<html lang="en">
<head>
  <script defer>
```

```

// Simulate an asynchronous request to API 1
function fetchDataFromAPI1() {
  return new Promise( (resolve, reject) => {
    {
      setTimeout(() => {
        console.log("Fetched data from API 1");
        resolve("Data from API 1");
      }, 2000);
    }
  });
}

// Simulate an asynchronous request to API 2
function fetchDataFromAPI2(data1) {
  return new Promise( (resolve, reject) => {
    {
      setTimeout(() => {
        console.log("Fetched data from API 2, based on " + data1);
        resolve("Data from API 2, based on " + data1);
      }, 2000);
    }
  });
}

function fetchData()
{
  // Fetch data sequentially using callbacks
  fetchDataFromAPI1()
  .then(data1 => fetchDataFromAPI2(data1))
  .then(data2 => console.log(data2))
  .catch(error => console.log(error));
}

async function fetchData2()
{
  // Fetch data sequentially using callbacks
  try {
    data1 = await fetchDataFromAPI1();
    data2 = await fetchDataFromAPI2(data1);
    console.log(data2);
  }
  catch (error)
  {
    console.log(error);
  }
}

</script>
</head>
<body>
  <button onclick="fetchData()">ClickMe</button><br>
  <button onclick="fetchData2()">ClickMe2</button><br>
</body>
</html>

```

Output –

Experiment # 11

JavaScript Asynchronous Programming to get the weather info from openweathermap.org

Aim –

1. Familiarize with asynchronous web API **fetch**.

Sample code –

Javascript code with Promise, Then syntax -

```
<!-- https://openweathermap.org/
SignIn / Create Your Account, Confirm from your mail inbox.
MeControl -> My API Keys. Copy the key and use below -->

<!DOCTYPE html>
<html>
  <head>
    <script>
      const APIKey = '69fb4b02358c349120c314202ebb13ac';

      function getWeatherInfo()
      {
        let inputCity = document.getElementById('textbox1').value;
        console.log(inputCity);

        const apiUrl =
`https://api.openweathermap.org/data/2.5/weather?q=${inputCity}&units=metric&lang=en&appid=${APIKey}`;

        fetch(apiUrl)
          .then(response => response.json()) // response is complete http response with status code, headers and
body. response.json() parses the body as JSON, returns as Promise with json data
          .then(data => {
            console.log(data);

            let city = data.name;
            let temp = data.main.temp;
            let desc = data.weather[0].description;

            document.getElementById('cityName').textContent = `${city}`;
            document.getElementById('temperature').textContent = `${temp}`;
            document.getElementById('description').textContent = `${desc}`;
          })
          .catch(reason => {console.log(`Reason is : ${reason}`);})
          .finally(() => {console.log('Execution Completed');})
      }
    </script>
  </head>
  <body>
    <input type="text" id="textbox1" placeholder="Enter City Name" style="margin-right: 5px;">
    <button name="GetWeatherInfo" id="button1" onclick="getWeatherInfo()">Get Weather Info</button>
    <br>

    <h1>Weather Info</h1>
    <div id="weather">
      <h2>City: <label id="cityName"></label></h2>
      Temperature: <label id="temperature"></label>
      <br>
      Weather Description: <label id="description"></label>
    </div>
  </body>
</html>
```

Javascript code with Promise, Async/Await syntax -

```
<!-- https://openweathermap.org/
```

SignIn / Create Your Account, Confirm from your mail inbox.
MeControl -> My API Keys. Copy the key and use below -->

```
<!DOCTYPE html>
<html>
  <head>
    <script>
      const APIKey = '69fb4b02358c349120c314202ebb13ac';

      async function getWeatherInfo()
      {
        let inputCity = document.getElementById('textbox1').value;
        console.log(inputCity);

        const apiUrl =
`https://api.openweathermap.org/data/2.5/weather?q=${inputCity}&units=metric&lang=en&appid=${APIKey}`;

        try{
          let response = await fetch(apiUrl);
          let data = await response.json(); // response is complete http response with status code, headers and
body. response.json() parses the body as JSON, returns as Promise with json data
          console.log(data);

          let city = data.name;
          let temp = data.main.temp;
          let desc = data.weather[0].description;

          document.getElementById('cityName').textContent = `${city}`;
          document.getElementById('temperature').textContent = `${temp}`;
          document.getElementById('description').textContent = `${desc}`;
        }
        catch (error)
        {
          console.log(`Reason is : ${error}`);
        }
        finally
        {
          console.log('Execution Completed');
        }
      }
    </script>
  </head>
  <body>
    <input type="text" id="textbox1" placeholder="Enter City Name" style="margin-right: 5px;">
    <button name="GetWeatherInfo" id="button1" onclick="getWeatherInfo()">Get Weather Info</button>
    <br>

    <h1>Weather Info</h1>
    <div id="weather">
      <h2>City : <label id="cityName"></label></h2>
      Temperature : <label id="temperature"></label>
      <br>
      Weather Description: <label id="description"></label>
    </div>
  </body>
</html>
```

Output –

Weather Info

City : Hyderabad

Temperature : 29.76

Weather Description: broken clouds

Experiment # 12

JavaScript Asynchronous Programming to get the weather info from openweathermap.org & information in the form of a graph/chart.

Aim –

1. Familiarize with asynchronous web API **fetch** & Chart.js for various charts/graphs

Sample code –

```
<!-- https://openweathermap.org/
SignIn / Create Your Account, Confirm from your mail inbox.
MeControl -> My API Keys. Copy the key and use below -->

<!DOCTYPE html>
<html>
  <head>
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
    <script>
      const APIKey = '69fb4b02358c349120c314202ebb13ac';

      let tempArr = [];
      let chartLabels = [];
      let chartCreated = false;
      let myChart;

      function getCurTime()
      {
        const currentDate = new Date();

        // Get the current time as per the local time zone
        const hours = currentDate.getHours(); // Hours (0-23)
        const minutes = currentDate.getMinutes(); // Minutes (0-59)
        const seconds = currentDate.getSeconds(); // Seconds (0-59)

        // Format the time as HH:mm:ss
        const formattedTime = `${hours}:${minutes}:${seconds}`;
        return formattedTime;
      }

      async function getWeatherInfo()
      {
        let inputCity = document.getElementById('textbox1').value;
        console.log(inputCity);

        const apiUrl =
`https://api.openweathermap.org/data/2.5/weather?q=${inputCity}&units=metric&lang=en&appid=${APIKey}`;

        try{
          let response = await fetch(apiUrl);
          let data = await response.json(); // response is complete http response with status code, headers and
body. response.json() parses the body as JSON, returns as Promise with json data
          console.log(data);

          let city = data.name;
          let temp = data.main.temp;
          let desc = data.weather[0].description;

          document.getElementById('cityName').textContent = `${city}`;
          document.getElementById('temperature').textContent = `${temp}`;
          document.getElementById('description').textContent = `${desc}`;

          tempArr.push(temp);
          chartLabels.push(getCurTime());

          if (!chartCreated)
          {
```

```

        myChart = new Chart(document.getElementById('MyCanvas1').getContext('2d'), {
            type: 'bar',
            data: {
                labels: chartLabels,
                datasets: [{
                    data: tempArr,
                    borderColor: 'red',
                    borderWidth: 2
                }]
            }
        });
        chartCreated = true;
    }
    else
    {
        myChart.update();
    }

}
catch (error)
{
    console.log(`Reason is : ${error}`);
}
finally
{
    console.log('Execution Completed');
}
}

function getWeatherInfoPeriodically()
{
    setInterval(getWeatherInfo, 5000);
}
</script>
</head>
<body>
    <input type="text" id="textbox1" placeholder="Enter City Name" style="margin-right: 5px;">
    <button name="GetWeatherInfo" id="button1" onclick="getWeatherInfoPeriodically()">Get Weather
Info</button>
    <br>

    <h1>Latest Weather Info</h1>
    <div id="weather">
        <h2>City : <label id="cityName"></label></h2>
        Temperature : <label id="temperature"></label>
        <br>
        Weather Description: <label id="description"></label>
    </div>

    <h1>Weather Trend</h1>
    <canvas id="MyCanvas1" width="300" height="150"></canvas>
</body>
</html>

```

Output –

Get Weather Info

Latest Weather Info

City : Delhi

Temperature : 31.46
Weather Description: few clouds

Weather Trend



Experiment # 13

Standalone Java application with JDBC connection to RDBMS

Aim –

1. Familiarize with JDBC connections, to persist the data into database systems

Sample code –

```
import java.sql.*;

public class JdbcExample {

    // Database URL, username, and password
    // static final String DB_URL = "jdbc:mysql://localhost:3306/testdb";
    static final String DB_URL = "jdbc:sqlite:D:/SQLite/DBs/Student2DB";
    static final String USER = "root";
    static final String PASS = "password";

    public static void main(String[] args) {
        // Establish connection to the database
        try (Connection conn = DriverManager.getConnection(DB_URL/*, USER, PASS*/)) {

            // 1. Insert
            String insertSQL = "INSERT INTO Students2 (rollno, name, age, marks) VALUES (?, ?, ?, ?)";
            try (PreparedStatement pstmt = conn.prepareStatement(insertSQL)) {
                pstmt.setInt(1, 5);
                pstmt.setString(2, "krishna2");
                pstmt.setInt(3, 31);
                pstmt.setFloat(4, 91);
                int rowsInserted = pstmt.executeUpdate();
                System.out.println(rowsInserted + " row(s) inserted.");
                pstmt.close();
            }

            // 2. Select
            String selectSQL = "SELECT * FROM Students2";
            try (Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery(selectSQL)) {

                System.out.println("\nStudents2 in the database:");
                while (rs.next()) {
                    int id = rs.getInt("rollno");
                    String name = rs.getString("name");
                    int age = rs.getInt("age");
                    float marks = rs.getFloat("marks");
                    System.out.println("RollNo: " + id + ", Name: " + name + ", Age: " + age + ", Marks: " + marks);
                }

                stmt.close();
                rs.close();
            }

            conn.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Output –

Ensure record got inserted into backend database.
Ensure able to retrieve back that record from database.

Experiment # 14

XML for bookstore/library. Validate with XSD

Aim –

1. Familiarize with XML data exchange format.
2. Familiarize with XSD, for defining the document schema.
3. Validate XML with XSD and understand mistakes in XML, if any.

Sample code –

XML –

```
<?xml version="1.0" encoding="UTF-8"?>
<library xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="library.xsd">
  <book id="1">
    <title>Learning XML</title>
    <author>Jane Smith</author>
    <price>39.99</price>
    <publisher>Tech Press</publisher>
  </book>
  <book id="2">
    <title>Advanced XML</title>
    <author>John Doe</author>
    <price>49.99</price>
  </book>
</library>
```

XSD –

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <!-- Define a simple type for the price -->
  <xs:simpleType name="priceType">
    <xs:restriction base="xs:decimal">
      <xs:minInclusive value="35"/>
    </xs:restriction>
  </xs:simpleType>

  <!-- Define the "book" element -->
  <xs:complexType name="bookType">
    <xs:sequence>
      <xs:element name="title" type="xs:string"/>
      <xs:element name="author" type="xs:string"/>
      <xs:element name="price" type="priceType"/>
      <xs:element name="publisher" type="xs:string" minOccurs="0"/> <!-- Optional -->
    </xs:sequence>
    <xs:attribute name="id" type="xs:int" use="required"/>
  </xs:complexType>

  <!-- Define the "library" element, which can contain multiple "book" elements -->
  <xs:element name="library">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="book" type="bookType" maxOccurs="unbounded"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

Online XML validation tool -

<https://www.xmlvalidation.com/index.php?id=1&L=0>

Some Viva questions on JavaScript, JDBC, XML

JavaScript

1. How Javascript helps in client web pages?
2. What data types are supported in Javascript?
3. Where to define Javascript functions and how can we invoke them from html elements in web page?
4. What is the difference between == and === operators?
5. What is coalescing operator?
6. How do we use arrays in Javascript?
7. Give regular expression for checking the email format.
8. What all storage options we have with Javascript, on client side?
9. What is the root element for the html document in DOM model? What is the root element for the entire browser? What is the function to show some error message to the user in a dialog box?
10. Which Javascript engine is most popular in market?
11. How is arrow function defined in Javascript?
12. What is the difference between synchronous programming and asynchronous programming?
13. What are Promises in Javascript? What are the states of Promise?
14. What is the benefit of using Promise over callbacks?
15. What are async and await? Why and how they are used?
16. What is the web API for making network calls from Javascript?

JDBC

1. JDBC stands for? What is its purpose?
2. Name any 3 main classes used in JDBC connection to Database.
3. What is the difference between Statement, PreparedStatement and CallableStatement?

XML

1. What does XML stand for?
2. What is the purpose of XML? Where all it is used?
3. Any other popular data exchange format?
4. Name any two data exchange formats?
5. In what all ways, we can define XML document structure?
6. What is the way to define XML document structure? Like what should be the root element, what child elements it can contain...etc.

Apache Tomcat server setup

Download the Tomcat .zip file from <https://tomcat.apache.org/download-90.cgi>

Unzip/Extract it to C drive or D drive.

Ensure something like D:\apache-tomcat-9.0.102\apache-tomcat-9.0.102 present with bin folder in it.

Set system environment variable CATALINA_HOME to D:\apache-tomcat-9.0.102\apache-tomcat-9.0.102

In conf\tomcat-users.xml file, inside the tomcat-users element, add the following

```
<!-- Define roles -->
```

```
<role rolename="manager-gui"/>
```

```
<role rolename="admin-gui"/>
```

```
<!-- Define users with roles -->
```

```
<user username="admin" password="admin" roles="manager-gui,admin-gui"/>
```

Java Development Kit

Download and install JDK.

Set system environment variable JAVA_HOME to C:\Program Files\Java\jdk-23 (the folder wherever JDK got installed).

If the system has only JRE installed, instead of complete JDK, then set the environment variable JRE_HOME to C:\Program Files (x86)\Java\jre1.8.0_441 (the folder wherever JRE got installed)

Start Apache Tomcat server

Go to D:\apache-tomcat-9.0.102\apache-tomcat-9.0.102\bin folder. Run startup.bat

If security consent comes - Allow it.

In browser, go to <http://localhost:8080/>

If Apache page appears correctly, then setup is completed.

New Project in NetBeans IDE & Deploy the Server Application

Launch NetBeans.

New Project -> Java Web Application -> Give some project name, select some folder, where project files have to be placed. -> Add Apache tomcat server with admin user -> Finish.

In the project created -> Web Pages -> Right-click & New -> JSP...

Add the JSP code.

Run -> Build.

If Build fails with some WAR related error, check the POM file for the maven-war-plugin

Ramakrishna N - MGIT

version. Set it to 3.3.1.

To deploy to Apache Tomcat.....

Go to <http://localhost:8080/manager/html>

Login with admin/admin

Deploy the .war file built above.

Go to D:\apache-tomcat-9.0.102\apache-tomcat-9.0.102\webapps folder, our project should appear here.

In the project created -> Web Pages -> Right-click & New -> Servlet...

Give Servlet name. Complete the wizard.

Add Servlet's java code.

Run -> Build the code again.

Undeploy and Deploy the code from Tomcat Manager portal again.

Experiment # 15

Sample Dynamic web page with JSP Scriptlets and JSP Tag Library

Aim –

1. Familiarize with Web Server, Application Server
2. Familiarize with Java Server side technology – JSP
3. JSP Scriptlets, JSP Expressions, JSP Tag Library, JSP Expression Language

Sample code –

```
<!-- JSP directives -->
<%@ page contentType="text/html" pageEncoding="UTF-8" language="Java" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>

<!DOCTYPE html>
<html>
  <head>
    <title>JSP Basic Example</title>
  </head>
  <body>
    <h1>JSP Hello World!</h1>

    <p>
      <h2>Using JSP scriptlets</h2>

      <%
        String param1 = request.getParameter("param1");
        String description;

        if (param1.length() < 5)
        {
          description = "Very short value passed for param1";
        }
        else
        {
          description = "param1 length is " + param1.length();
        }
      %>

      <b>Parameter in request is: <%= request.getParameter("param1").toString() %> </b>
      <b>Description: <%= description %> </b>
    </p>

    <br>

    <p>
      <h2>Using JSP Expression Language and JSP Tag Library</h2>
      <b>Parameter in request is: ${param.param1} </b>

      <c:choose>
        <c:when test="${param.param1.length() < 5}">
          <b>Description: Very short value passed for param1 </b>
        </c:when>
        <c:otherwise>
          <b>Description: param1 length is ${param.param1.length()} </b>
        </c:otherwise>
      </c:choose>
    </p>

  </body>
</html>
```

Output –



Experiment # 16

Student data form submission with JSP and Servlets. Persist data into RDBMS, with JDBC.

Aim –

1. JSP vs Servlets usage
2. Understanding HTTP protocol – types, resource path, query params, headers, body content.

Sample code –

JSP file –

```
<%@ page contentType="text/html" pageEncoding="UTF-8" language="java" %>
<!DOCTYPE html>
<html>
  <head>
    <title>Student Information Form</title>
  </head>
  <body>
    <h2>Student Information Form</h2>
    <form action="StudentInfoSubmit" method="post">
      <label for="name">Name:</label>
      <input type="text" id="name" name="name" required /><br><br>

      <label for="rollNo">Roll No:</label>
      <input type="text" id="rollNo" name="rollNo" required /><br><br>

      <label for="age">Age:</label>
      <input type="number" id="age" name="age" required /><br><br>

      <input type="submit" value="Submit" />
    </form>
  </body>
</html>
```

Java Servlet file –

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet(urlPatterns = {"/StudentInfoSubmit"})
public class StudentInfoSubmit extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Retrieve form data from the request
        String name = request.getParameter("name");
        String rollNo = request.getParameter("rollNo");
        String age = request.getParameter("age");

        // Use JDBC and save this student info into RDBMS.

        // Set the response content type
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Output the received data
        out.println("<html><body>");
```

```
        out.println("<h2>Student Information Submitted:</h2>");
        out.println("<p><strong>Name:</strong> " + name + "</p>");
        out.println("<p><strong>Roll No:</strong> " + rollNo + "</p>");
        out.println("<p><strong>Age:</strong> " + age + "</p>");
        out.println("</body></html>");
    }
}
```

Output –

The first screenshot shows a web browser at the URL `localhost:8080/StudentForm-1.0-SNAPSHOT/StudentDataForm.jsp`. The page title is "Student Information Form". It contains three input fields: "Name:" with the value "krishna", "Roll No:" with the value "5", and "Age:" with the value "32". Below these fields is a "Submit" button.

The second screenshot shows the browser at the URL `localhost:8080/StudentForm-1.0-SNAPSHOT/StudentInfoSubmit`. The page title is "Student Information Submitted:". It displays the submitted information in a list-like format: "Name: krishna", "Roll No: 5", and "Age: 32".

Experiment # 17

Student data submission with JavaScript Async Programming at client side & JSP and Servlets at server side. Persist data into RDBMS, with JDBC.

Aim –

1. Role of Javascript client side validations, before submitting the data to server side.
2. Http network calls with Javascript web API fetch.
3. Sending the data in JSON format, instead of html formdata, to the server side.
4. JSON parsing of data in Servlet, for persisting the same into RDBMS.

Sample code –

JSP file (including Javascript) –

```
<%@ page contentType="text/html" pageEncoding="UTF-8" language="java" %>
<!DOCTYPE html>
<html>
  <head>
    <title>Student Information Form</title>

    <script>
      function SubmitTheForm()
      {
        console.log("Inside js");

        let formData = new Object();
        formData.name = document.getElementById('nameId').value;
        formData.rollNo = document.getElementById('rollNoId').value;
        formData.age = document.getElementById('ageId').value;

        console.log(JSON.stringify(formData));

        fetch("StudentInfoSubmit", {
          method: "POST",
          headers: {
            "Content-Type": "application/json" // Send as JSON
          },
          body: JSON.stringify(formData) // Convert to JSON string
        })
        .then(response => response.text())
        .then(html => {
          document.open();
          document.write(html); // Replace current page with new HTML
          document.close();
        })
        .catch(error => console.error("Error:", error));
      }
    </script>
  </head>
  <body>
    <h2>Student Information Form</h2>

    <label for="name">Name:</label>
    <input type="text" id="nameId" name="name" required /><br><br>

    <label for="rollNo">Roll No:</label>
    <input type="text" id="rollNoId" name="rollNo" required /><br><br>

    <label for="age">Age:</label>
    <input type="number" id="ageId" name="age" required /><br><br>

    <button onclick="SubmitTheForm()">Submit</button>
  </body>
</html>
```

Java Servlet file –

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.json.JSONObject;

@WebServlet(urlPatterns = {"/StudentInfoSubmit"})
public class StudentInfoSubmit extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // Read JSON from request body
        BufferedReader reader = request.getReader();
        StringBuilder jsonString = new StringBuilder();
        String line;
        while ((line = reader.readLine()) != null) {
            jsonString.append(line);
        }

        // Parse JSON string
        JSONObject jsonObject = new JSONObject(jsonString.toString());
        String name = jsonObject.getString("name");
        String rollNo = jsonObject.getString("rollNo");
        String age = jsonObject.getString("age");

        // Use JDBC and save this student info into RDBMS.

        // Set the response content type
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Output the received data
        out.println("<html><body>");
        out.println("<h2>Student Information Submitted:</h2>");
        out.println("<p><strong>Name:</strong> " + name + "</p>");
        out.println("<p><strong>Roll No:</strong> " + rollNo + "</p>");
        out.println("<p><strong>Age:</strong> " + age + "</p>");
        out.println("</body></html>");
    }
}
```

Output –

←

↻

i

localhost:8080/StudentForm2-1/StudentDataForm.jsp

Student Information Form

Name:

krishna

Roll No:

6

Age:

32

Submit

←

↻

i

localhost:8080/StudentForm2-1/StudentDataForm.jsp

Student Information Submitted:

Name: krishna

Roll No: 6

Age: 32

Experiment # 18

Fetch the Student information that got persisted into RDBMS with previous experiments.

Aim –

1. JSP vs Servlet typical usage
2. MVC – Servlet acting as Controller, JSP acting as View, JDBC connection logic acting as Model.

Sample code –

Initial JSP file (FetchStudentInfo.jsp) –

```
<%@ page contentType="text/html" pageEncoding="UTF-8" language="java" %>
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Information</title>
</head>
<body>
  <h1>Fetch Student Information</h1>

  <!-- Form to input rollno and fetch student details -->
  <form action="FetchStudentInfo" method="get">
    <label for="rollno">Enter Roll No: </label>
    <input type="text" id="rollno" name="rollno" required>
    <button type="submit">Get Student Info</button>
  </form>
</body>
</html>
```

Java Servlet file (acting as Controller) –

```
package com.mycompany.fetchstudentinfo;

import java.io.IOException;
import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.util.logging.Logger;

@WebServlet(name = "FetchStudentInfoServlet", urlPatterns = {"/FetchStudentInfo"})
public class FetchStudentInfoServlet extends HttpServlet {
    private static final Logger logger = Logger.getLogger(FetchStudentInfoServlet.class.getName());

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Fetch the roll number from the request parameter
        String rollnoParam = request.getParameter("rollno");

        if (rollnoParam != null && !rollnoParam.isEmpty()) {
            try {
                int rollno = Integer.parseInt(rollnoParam);

                // Create an instance of StudentDAO and get the student details
                StudentDAO studentDAO = new StudentDAO(logger);
                Student student = studentDAO.getStudentByRollno(rollno);

                // Set the student object in the request scope
                if (student != null) {
```

```

        request.setAttribute("student", student);
    } else {
        request.setAttribute("message", "No student found with this roll number: " + rollno);
    }

    } catch (NumberFormatException e) {
        // If the rollno is not a valid number, set an error message
        request.setAttribute("message", "Invalid roll number.");
    }
    } else {
        request.setAttribute("message", "Roll number is required.");
    }

    // Forward the request to the JSP page
    RequestDispatcher dispatcher = request.getRequestDispatcher("/StudentInfo.jsp");
    dispatcher.forward(request, response);
}
}

```

Java files with JDBC logic (acting as Model) –

StudentDAO.java

```

package com.mycompany.fetchstudentinfo;

import java.sql.*;
import java.util.logging.Logger;

public class StudentDAO {
    private static Logger logger;

    public StudentDAO(Logger logger)
    {
        this.logger = logger;
    }

    public Student getStudentByRollno(int rollnum) {
        Student student = null;
        Connection conn = null;
        PreparedStatement ps = null;
        ResultSet rs = null;

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            // Step 1: Establish database connection
            conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/testdb", "root", "");

            // Step 2: Prepare the SQL query
            String sql = "SELECT * FROM students WHERE rollno = ?";
            ps = conn.prepareStatement(sql);
            ps.setInt(1, rollnum); // Set the rollno in the query

            // Step 3: Execute the query
            rs = ps.executeQuery();

            // Step 4: Process the result set
            if (rs.next()) {
                int id = rs.getInt("rollno");
                String name = rs.getString("name");
                int age = rs.getInt("age");
                student = new Student(id, name, age); // Create a student object with fetched data
            }
        }
        catch (ClassNotFoundException e)
    }
}

```

```

    {
        logger.warning("NRK8: ClassNotFoundException" + e);
    }
    catch (SQLException e) {
        logger.warning("NRK1: exception from JDBC" + e);
    } finally {
        try {
            if (rs != null) rs.close();
            if (ps != null) ps.close();
            if (conn != null) conn.close();
        } catch (SQLException e) {
            logger.warning("NRK2: exception from JDBC" + e);
        }
    }

    return student;
}
}

```

Student.java

```

package com.mycompany.fetchstudentinfo;

public class Student {
    private int rollno;
    private String name;
    private int age;

    public Student(int rollno, String name, int age) {
        this.rollno = rollno;
        this.name = name;
        this.age = age;
    }

    public int getRollno() {
        return rollno;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }
}

```

JSP file (acting as View) –

```

<%@ page contentType="text/html" pageEncoding="UTF-8" language="java" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Student Information</title>
</head>
<body>
    <h1>Student Information</h1>

    <!-- Display message if no student found or if there's any error -->
    <c:if test="${not empty message}">
        <p>${message}</p>
    </c:if>

```

```
<!-- Display student details if student object is not null -->
<c:if test="${not empty student}">
  <table border="1">
    <tr>
      <th>Roll No</th>
      <th>Name</th>
      <th>Age</th>
    </tr>
    <tr>
      <td>${student.rollno}</td>
      <td>${student.name}</td>
      <td>${student.age}</td>
    </tr>
  </table>
</c:if>
</body>
</html>
```

Pom.xml

```
<dependencies>
  <dependency>
    <groupId>javax</groupId>
    <artifactId>javaee-web-api</artifactId>
    <version>6.0</version>
    <scope>provided</scope>
  </dependency>

  <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.26</version>
  </dependency>

  <dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>jstl</artifactId>
    <version>1.2</version>
  </dependency>
</dependencies>
```

Output –

 localhost:8080/FetchStudentInfo-1/FetchStudentInfo.jsp

Fetch Student Information

Enter Roll No:

Experiment # 19

Session tracking with Cookies.

Aim –

1. Usage of Cookies, for tracking the client session, by server.
2. How browser saves the Cookies and how it gets sent back to the server in every subsequent calls to the server.

Sample code –

JSP file (for sending the cookie to the client initially) –

```
<%@ page contentType="text/html" pageEncoding="UTF-8"%>

<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>JSP Page for Cookie Example</title>
  </head>
  <body>
    <h1>Cookie Example</h1>

    <%
      Cookie cookie1 = new Cookie("ClientState", "PlacedOrder");
      response.addCookie(cookie1);
    %>

    <form method="post" action="SubmitADummyForm">
      <input type="text" name="Text1">
      <button>Submit</button>
    </form>

  </body>
</html>
```

Java Servlet file (receiving another request from client, along with Cookie) –

```
@WebServlet(name = "ServletExample1", urlPatterns = {"/SubmitADummyForm"})
public class ServletExample1 extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html; charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            out.println("<!DOCTYPE html>");
            out.println("<html>");
            out.println("<head>");
            out.println("<title>Servlet ServletExample1</title>");
            out.println("</head>");

            out.println("<body>");
            out.println("<h2>User given input is: " + request.getParameter("Text1") + "</h2><br>");
            out.println("<h2>Cookies are:</h2>");
            Cookie[] cookies = request.getCookies();
            if (cookies != null)
            {
                for (Cookie cookie : cookies)
                {
                    out.println("<h3>" + cookie.getName() + ": " + cookie.getValue() + "</h3><br>");
                }
            }
            out.println("</body>");
        }
    }
}
```

```
        out.println("</html>");  
    }  
}  
}
```

Output –

← ↻ ⓘ localhost:8080/SessionTrackingWithCookie-1/InitialWebPage.jsp

Cookie Example

← ↻ ⓘ localhost:8080/SessionTrackingWithCookie-1/SubmitADummyForm

User given input is: Ramakrishna

Cookies are:

JSESSIONID: DCCCE1AE44630731214CE7A7F964A119

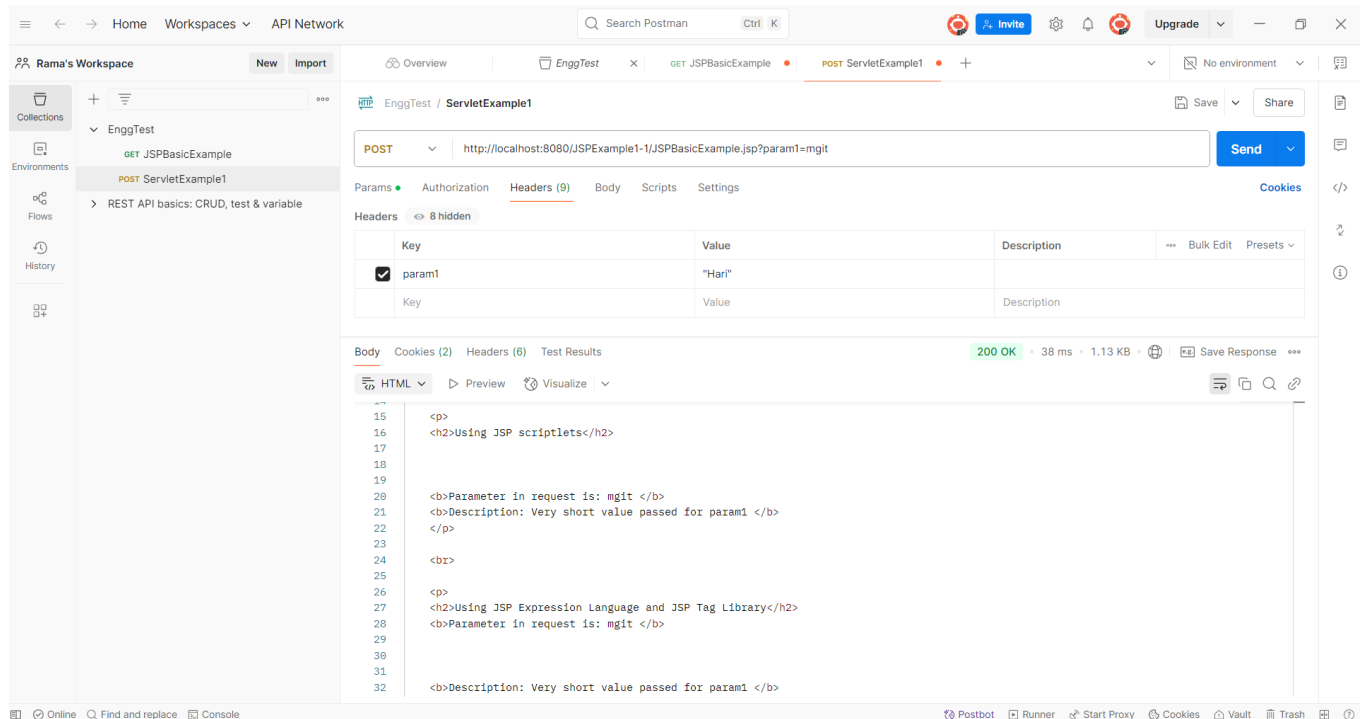
ClientState: PlacedOrder

Postman

Usage of Postman for Server side validations, without needing to build client side applications.

Download and Install Postman client application

Fire HTTP requests directly to the Server applications. Tweak input parameters and validate server response for each of the inputs from client.



Some Viva questions on Java Server Technologies – JSP, Servlets

Server Applications, JSP and Servlets

1. What is the difference between a Web Server and Application Server?
2. Brief on HTTP protocol. How is it used in web technologies?
3. What is the difference between JSP and Servlets? Which one is preferable in what situation?
4. What are different types of scripting elements in JSP?
5. What are JSP directives?
6. What is JSP Tag Library? Give some examples.
7. What is the base/main class for Servlets, that needs to be extended for custom server application?
8. How can we access the input parameters that client has passed to the server, in JSP and Servlets?
9. Name any Session tracking mechanism.
10. What are Cookies? What purpose do they solve?
11. What is RequestDispatcher?

GitHub & Git

<https://github.com/>

Create Your own account

Create a Repository, for the project, that you would like to upload/track revision history. Choose Public / Private scope for the Repository, during the creation.

Option 1 – Upload the project files directly.

Option 2 – Use git client tool to push/commit your changes and let the revision history tracked correctly.

[This is the option used in real world, when multiple people work on the same project. Final Pull Request need approval from team mates, to merge your changes into server repository]

Install git client tool, for your Windows platform. (<https://git-scm.com/downloads/win>)

Open cmd prompt. Run the following commands:

```
git init <projectname>
```

```
cd into that folder
```

```
git clone "https://github.com/<YourAccount>/<YourRepository>"
```

- When it asks for Your account credentials, provide the same with browser login option.

```
git status → This should show the below:
```

```
On branch main
```

```
Your branch is up to date with 'origin/main'.
```

```
nothing to commit, working tree clean
```

```
git branch
```

```
Lists all the existing branches and also shows which is current active one.
```

```
git branch <newbranch>
```

```
git checkout <newbranch> // to switch to the branch
```

```
// Edit some files now.
```

```
git status → This should show the files in the project that you have edited above.
```

```
git add <files changes>
```

```
This is referred as staged.
```

```
Whenever we modify the files, we need to add them again to branch.
```

```
git commit -m "Some comment describing the change to ease understanding"
```

```
git push
```

```
git push --set-upstream origin <newbranch>
```

Check the repository in github now. It should have the new branch created over there with your file change.

Raise Pull Request from the server. Merge it with main branch, if there are no conflicts.

Check your code now in the github server. It should have your latest change.

Node JS Setup

Download and Install Node and NPM from <https://nodejs.org>

Cmd prompt, run

```
C:\Users\Rama>node -v  
v22.14.0
```

```
C:\Users\Rama>npm -v  
10.9.2
```

If the above shows, versions, then installation is successful.

For New Project....

Create a folder in Windows.

Cd to that folder from cmd prompt

Npm init -y // This will initialize a new Node project structure as needed, with default settings.

Add the .js file as per the server application needed.

To Run it....

Cmd prompt, run

```
Node <YourApp>.js
```

To install any additional Node Modules....

Npm install <Module>

For example...

Npm install express

To check / search for modules available... <https://www.npmjs.com/>

Experiment # 20

Node JS – Sample Server application with http module

Aim –

1. Familiarize with Node JS environment
2. Create custom server application with http module

Sample code –

```
HttpSample1.js

const http = require('http');
const url = require('url');

const server = http.createServer((req, res) => {
  const parsedUrl = url.parse(req.url, true); // Parse the URL and query parameters
  const query = parsedUrl.query;

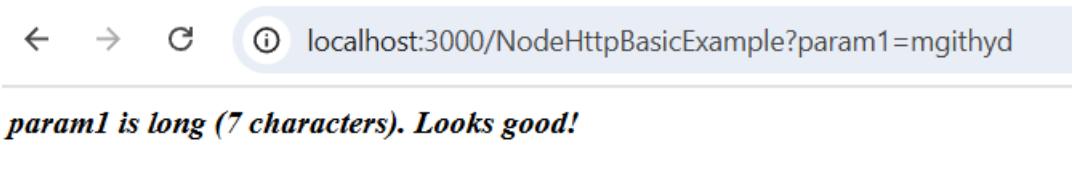
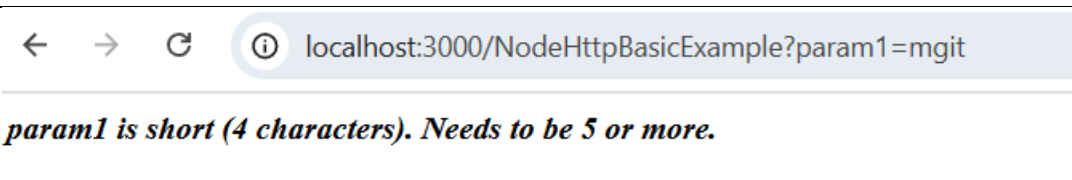
  if (req.method === 'GET' && parsedUrl.path.startsWith('/NodeHttpBasicExample'))
  {
    const param1 = query.param1; // Extract param1

    res.writeHead(200, { 'Content-Type': 'text/html' });

    if (param1 === undefined) {
      res.end('<b><I>param1 is missing in the request.</I></b>');
    } else if (param1.length < 5) {
      res.end('<b><I>param1 is short (${param1.length} characters). Needs to be 5 or more.</I></b>');
    } else if (param1.length >= 5) {
      res.end('<b><I>param1 is long (${param1.length} characters). Looks good!</I></b>');
    }
  }
  else
  {
    res.writeHead(400, { 'Content-Type': 'text/html' });
    res.end('<b><I>Unsupported request from client</I></b>');
  }
});

const PORT = 3000;
server.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

Output –

Experiment # 21

Node JS – Sample Server application with ExpressJS module

Aim –

1. Familiarize with Express module
2. Create custom server application with Express

Sample code –

From cmd prompt, post creating the project structure with npm, run the following to let express module get installed/added.

```
npm install express
```

```
ExpressSample1.js
```

```
const express = require('express');  
const app = express();
```

```
app.get('/ExpressBasicExample', (req, res) => {  
  const param1 = req.query.param1;
```

```
  res.set('Content-Type', 'text/html');
```

```
  if (param1 === undefined) {  
    res.status(200).send('<b><i>param1 is missing in the request.</i></b>');  
  } else if (param1.length < 5) {  
    res.status(200).send(`<b><i>param1 is short (${param1.length} characters). Needs to be 5 or  
more.</i></b>`);  
  } else if (param1.length >= 5) {  
    res.status(200).send(`<b><i>param1 is long (${param1.length} characters). Looks good!</i></b>`);  
  }  
});
```

```
// Catch all other unsupported routes
```

```
app.use((req, res) => {  
  res.status(400).send('<b><i>Unsupported request from client</i></b>');  
});
```

```
const PORT = 3001;  
app.listen(PORT, () => {  
  console.log(`Express server running at http://localhost:${PORT}`);  
});
```

Output –

← → ↻ ⓘ localhost:3001/ExpressBasicExample?param1=mgithyd

param1 is long (7 characters). Looks good!

← → ↻ ⓘ localhost:3001/ExpressBasicExample?param1=mgit

param1 is short (4 characters). Needs to be 5 or more.

Experiment # 22

Node JS – Express application with REST API for CRUD operations over Students data

Aim –

1. Using Express for creating a web application exposing APIs for Creating, Reading, Updating, Deleting Students data.
2. Familiarize that client side can be native applications expecting data in formats like json.

Sample code –

From cmd prompt, post creating the project structure with npm, run the following to let express module get installed/added.

npm install express

StudentsCRUDapi.js

```
const express = require('express');
const app = express();
```

```
app.use(express.json()); // Middleware to parse JSON request bodies
```

```
let students = []; // In-memory "database"
let nextId = 1; // Auto-incrementing ID
```

```
// Create a new student
app.post('/students', (req, res) => {
  const { name, age } = req.body;
  if (!name || !age) {
    return res.status(400).json({ error: 'Name and age are required.' });
  }
}
```

```
  const student = { id: nextId++, name, age };
  students.push(student);
  res.status(201).json(student);
});
```

```
// Get all students
app.get('/students', (req, res) => {
  res.json(students);
});
```

```
// Get a single student by ID
app.get('/students/:id', (req, res) => {
  const student = students.find(s => s.id === parseInt(req.params.id));
  if (!student) return res.status(404).json({ error: 'Student not found.' });
  res.json(student);
});
```

```
// Update a student by ID
app.put('/students/:id', (req, res) => {
  const student = students.find(s => s.id === parseInt(req.params.id));
  if (!student) return res.status(404).json({ error: 'Student not found.' });
```

```
  const { name, age } = req.body;
  if (name) student.name = name;
  if (age) student.age = age;
```

```
  res.json(student);
});
```

```
// Delete a student by ID
app.delete('/students/:id', (req, res) => {
  const index = students.findIndex(s => s.id === parseInt(req.params.id));
  if (index === -1) return res.status(404).json({ error: 'Student not found.' });
```

```

const removed = students.splice(index, 1);
res.json({ message: 'Student deleted.', student: removed[0] });
});

// Fallback for undefined routes
app.use((req, res) => {
  res.status(404).json({ error: 'Route not found.' });
});

// Start the server
const PORT = 3002;
app.listen(PORT, () => {
  console.log(`Student CRUD API is running at http://localhost:${PORT}`);
});

```

Output – (From Postman)

POST a new Student.

The screenshot shows the Postman interface with a workspace named 'EnggTest'. The selected request is 'POST StudentsCRUD1' with the URL 'http://localhost:3002/students'. The request body is a JSON object: `{ "name": "ramakrishna", "age": 33 }`. The response is a 201 status code, indicating 'Created', with a response time of 14 ms and a body size of 278 B. The response body is a JSON object: `{ "id": 2, "name": "ramakrishna", "age": 33 }`.

GET all existing Students

workspace

New Import

EnggTest

GET JSPBasicExample

GET JSPEExample1

POST StudentInfoSubmit

POST StudentsCRUD1

GET StudentsCRUD2

GET StudentsCRUD2a

PUT New Request

REST API basics: CRUD, test & variable

EnggTest / StudentsCRUD2

GET

http://localhost:3002/students

Send

Params

Authorization

Headers (6)

Body

Scripts

Settings

Cookies

Query Params

	Key	Value	Description	Bulk Edit
	Key	Value	Description	

Body

Cookies

Headers (7)

Test Results

200 OK • 12 ms • 307 B

Save Response

JSON

Preview

Visualize

```

1  [
2    {
3      "id": 1,
4      "name": "rama",
5      "age": 32
6    },
7    {
8      "id": 2,
9      "name": "ramakrishna",
10     "age": 33
11   }
12 ]

```

GET a particular Student

Workspace

New Import

EnggTest

GET JSPBasicExample

GET JSPEExample1

POST StudentInfoSubmit

POST StudentsCRUD1

GET StudentsCRUD2

GET StudentsCRUD2a

PUT New Request

REST API basics: CRUD, test & variable

EnggTest / StudentsCRUD2a

GET

http://localhost:3002/students/2

Send

Params

Authorization

Headers (6)

Body

Scripts

Settings

Cookies

Query Params

	Key	Value	Description	Bulk Edit
	Key	Value	Description	

Body

Cookies

Headers (7)

Test Results

200 OK • 8 ms • 273 B

Save Response

JSON

Preview

Visualize

```

1  {
2    "id": 2,
3    "name": "ramakrishna",
4    "age": 33
5  }

```

PUT / Update a particular student's detail

Workspace **New** **Import** Overview EnggTest GET JSPBasic GET JSPEXam POST StudentInfo POST Student GET Students GET Students PUT New Request + No environment

EnggTest / **New Request** Save Share

PUT http://localhost:3002/students/2 Send

Params Authorization Headers (8) **Body** Scripts Settings Cookies Beautify

none form-data x-www-form-urlencoded **raw** binary GraphQL JSON

```
1 { "name": "ramakrishna", "age": 40 }
```

Body Cookies Headers (7) Test Results 200 OK 12 ms 273 B Save Response

{ JSON Preview Visualize

```
1 {
2   "id": 2,
3   "name": "ramakrishna",
4   "age": 40
5 }
```

GET all students once, to check the above Update call.

Workspace **New** **Import** Overview EnggTest GET JSPBasic GET JSPEXam POST StudentInfo POST Student GET Students GET Students PUT New Request + No environment

EnggTest / **StudentsCRUD2** Save Share

GET http://localhost:3002/students Send

Params Authorization Headers (6) **Body** Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (7) Test Results 200 OK 14 ms 307 B Save Response

{ JSON Preview Visualize

```
1 [
2   {
3     "id": 1,
4     "name": "rama",
5     "age": 32
6   },
7   {
8     "id": 2,
9     "name": "ramakrishna",
10    "age": 40
11  }
12 ]
```

DELETE an existing student

Workspace **New** **Import** Overview EnggTest X GET JSPBat GET JSPEXa POST Studer POST Studer GET Student GET Student PUT Student DEL Studen + No environment

EnggTest / StudentsCRUD4 Save Share

DELETE http://localhost:3002/students/1 Send

Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (7) Test Results 200 OK 13 ms 307 B Save Response

JSON Preview Visualize

```
1 {
2   "message": "Student deleted.",
3   "student": {
4     "id": 1,
5     "name": "Iama",
6     "age": 32
7   }
8 }
```

GET all the students once, to check the above DELETE.

Workspace **New** **Import** Overview EnggTest GET JSPBat GET JSPEXa POST Studer POST Studer GET Student GET Student PUT Student DEL Studen + No environment

EnggTest / StudentsCRUD2 Save Share

GET http://localhost:3002/students Send

Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (7) Test Results 200 OK 8 ms 275 B Save Response

JSON Preview Visualize

```
1 [
2   {
3     "id": 2,
4     "name": "Iamakrishna",
5     "age": 40
6   }
7 ]
```